

1 LF Installation and Setup related challenges

Initially, I had tried setting up LF using the VS Code extension on Windows 11 with the following setup.

- Operating System - Windows 11
- VS Code - 1.103.2
- Java - 24.0.2
- Python - 3.13.5
- cmake - 4.1.0
- gcc and g++ - MinGW

On Windows, I encountered and tried mitigating several issues with C, C++ and Python targets which are summarized below.

1.1 My first LF build failed for all the 3 targets (C, C++ and Python)

On debugging, I realized that these issues were due to my GCC setup which used MinGW 32-bit. This resulted in a mismatch between 64-bit and 32-bit dependencies. Therefore, I installed and configured 64-bit MinGW.

```
gcc and g++ - - MinGW-W64
gcc (MinGW-W64 x86_64-msvcrt-posix-seh, built by Brecht Sanders, r1) 15.2.0
g++ (MinGW-W64 x86_64-msvcrt-posix-seh, built by Brecht Sanders, r1) 15.2.0
```

1.1.1 Trouble building C and Python targets

While the C++ target started building and working fine after switching to MinGW-64bit with the above setup, I continued having trouble building the C and Python targets, which rely on the C runtime.

Although the generation step worked fine, cmake failed while compiling the first file with the following error.

```

--- Executing command: cmake --build . --target install --parallel 8 --config Debug
[1/35] Building C object core/CMakeFiles/reactor-c.dir/mixed_radix.c.obj
FAILED: [code=1] core/CMakeFiles/reactor-c.dir/mixed_radix.c.obj
C:\MyApps\MinGW64\bin\gcc.exe -DINITIAL_EVENT_QUEUE_SIZE=10 -
DINITIAL_REACT_QUEUE_SIZE=10 -DLF_FILE_SEPARATOR="/" -
DLF_PACKAGE_DIRECTORY="c:\\MyData\\SelfStudy\\Programming\\SampleProgs\\lf\\myprojs
\\hello_world\\" -DLF_REACTION_GRAPH_BREADTH=1 -
DLF_SOURCE_DIRECTORY="c:\\MyData\\SelfStudy\\Programming\\SampleProgs\\lf\\myprojs\\
hello_world\\src\\" -
DLF_SOURCE_GEN_DIRECTORY="c:\\MyData\\SelfStudy\\Programming\\SampleProgs\\lf\\my
projs\\hello_world\\src-gen\\hw\\" -DLOG_LEVEL=2 -DNUMBER_OF_WATCHDOGS=0 -
DNUMBER_OF_WORKERS=0 -DPLATFORM_Windows -DSCHEDULER=SCHED_NP -
IC:/MyData/SelfStudy/Programming/SampleProgs/lf/myprojs/hello_world/src-
gen/hw/core/.../include -
IC:/MyData/SelfStudy/Programming/SampleProgs/lf/myprojs/hello_world/src-
gen/hw/core/.../include/core -
IC:/MyData/SelfStudy/Programming/SampleProgs/lf/myprojs/hello_world/src-
gen/hw/core/.../include/core/federated -
IC:/MyData/SelfStudy/Programming/SampleProgs/lf/myprojs/hello_world/src-
gen/hw/core/.../include/core/federated/network -
IC:/MyData/SelfStudy/Programming/SampleProgs/lf/myprojs/hello_world/src-
gen/hw/core/.../include/core/platform -
IC:/MyData/SelfStudy/Programming/SampleProgs/lf/myprojs/hello_world/src-
gen/hw/core/.../include/core/modal_models -
IC:/MyData/SelfStudy/Programming/SampleProgs/lf/myprojs/hello_world/src-
gen/hw/core/.../include/core/threaded -
IC:/MyData/SelfStudy/Programming/SampleProgs/lf/myprojs/hello_world/src-
gen/hw/core/.../include/core/utis -
IC:/MyData/SelfStudy/Programming/SampleProgs/lf/myprojs/hello_world/src-
gen/hw/core/federated/RTI -
IC:/MyData/SelfStudy/Programming/SampleProgs/lf/myprojs/hello_world/src-
gen/hw/trace/api/types -
IC:/MyData/SelfStudy/Programming/SampleProgs/lf/myprojs/hello_world/src-
gen/hw/version/api -
IC:/MyData/SelfStudy/Programming/SampleProgs/lf/myprojs/hello_world/src-
gen/hw/logging/api -
IC:/MyData/SelfStudy/Programming/SampleProgs/lf/myprojs/hello_world/src-gen/hw/tag/api -
IC:/MyData/SelfStudy/Programming/SampleProgs/lf/myprojs/hello_world/src-
gen/hw/low_level_platform/api -
IC:/MyData/SelfStudy/Programming/SampleProgs/lf/myprojs/hello_world/src-
gen/hw/platform/api -O3 -DNDEBUG -std=gnu11 -fdiagnostics-color=always /W4 -MD -MT
core/CMakeFiles/reactor-c.dir/mixed_radix.c.obj -MF core\\CMakeFiles\\reactor-
c.dir\\mixed_radix.c.obj.d -o core/CMakeFiles/reactor-c.dir/mixed_radix.c.obj -c
C:/MyData/SelfStudy/Programming/SampleProgs/lf/myprojs/hello_world/src-
gen/hw/core/mixed_radix.c
gcc.exe: warning: /W4: linker input file unused because linking not done
gcc.exe: error: /W4: linker input file not found: No such file or directory

```

1.1.2 Making C target work

On debugging further, I could isolate the root cause of error while building the C target.

The error was due to gcc interpreting /W4 as a linker input file rather than a compiler warning level flag. /W4 is used with Microsoft's Visual C++ compiler (cl.exe) to set the warning level to 4, but gcc does not recognize /W4. Instead, gcc uses -Wall, -Wextra, or similar flags to control warning levels. In the CMake gcc configuration generated by LF, the /W4 flag was being incorrectly passed to gcc, causing it to fail when it tries to treat /W4 as a file.

On searching through the files generated by LF, I noticed the following.

1. "<myprojs>\hello_world\src-gen\hw\core\lf_utils.cmake" defined the lf_enable_compiler_warnings function as follows.

```
function(lf_enable_compiler_warnings target)
    if(${CMAKE_SYSTEM_NAME} STREQUAL "Linux")
        target_compile_options(${target} PRIVATE -Wall -Wextra -Wpedantic -Werror)
    elseif(${CMAKE_SYSTEM_NAME} STREQUAL "Windows")
        target_compile_options(${target} PRIVATE /W4)
    elseif(${CMAKE_SYSTEM_NAME} STREQUAL "Darwin")
        target_compile_options(${target} PRIVATE -Wall)
    else()
        target_compile_options(${target} PRIVATE -Wall)
    endif()
endfunction()
```

For, cmake Windows, it added /W4 to the target_compile_options.

2. 1 above resulted in the following flags in "<myprojs>\hello_world\src-gen\hw\build\build.ninja" (MinGW64 gcc uses Ninja generator).

```
DEP_FILE = core\CMakeFiles\reactor-c.dir\mixed_radix.c.obj.d
FLAGS = -O3 -DNDEBUG -std=gnu11 -fdiagnostics-color=always /W4
```

For test the fix, I manually replaced "/W4" with "-Wall" in "build.ninja", and reran only the following failing cmake command from the failure log.

```
cmake --build . --target install --parallel 8 --config Debug
```

With this manual patch, the C target was built successfully for me. However, the Python target still had issues (covered in the following sub-section "Challenges in making Python target work on Windows")

1.1.3 Identifying the patch to be implemented by LF developers for C target

I had to figure out a way to tweak "target_compile_options" for the VS Code LF extension on Windows so that the generated files pass the right value (-Wall instead of /W4).

As an experiment, I first looked through the LF command-line interface (cli) "lfc" to see if there's a way to modify the target_compile_options, but could not find any. Therefore, starting backwards from "build.ninja", I studied the LF generated files for the C target.

Eventually, I made out that "lf_utils.cmake" was indeed the first file in LF generation flow where "target_compile_options" was being written, and required the following fix by LF developers for gcc on Windows.

Comparing files lf_utils.cmake.orig and lf_utils.cmake.updated

```
***** lf_utils.cmake.orig
elseif(${CMAKE_SYSTEM_NAME} STREQUAL "Windows")
    target_compile_options(${target} PRIVATE /W4)
elseif(${CMAKE_SYSTEM_NAME} STREQUAL "Darwin")
***** LF_UTILS.CMAKE.UPDATED
elseif(${CMAKE_SYSTEM_NAME} STREQUAL "Windows")
    if(${CMAKE_C_COMPILER} MATCHES "gcc\\.exe")
        target_compile_options(${target} PRIVATE -Wall)
    else()
        target_compile_options(${target} PRIVATE /W4)
    endif()
elseif(${CMAKE_SYSTEM_NAME} STREQUAL "Darwin")
*****
```

The above requirement was communicated to the LF developers by Prof Chattopadhyay who provided the patch. My C target also started working fine after consuming this patch.

1.1.4 Challenges in making Python target work on Windows

With the above patch, although the Python target also built without issues, it failed to run with the following error.

```
-
Traceback (most recent call last):
  File "<myprojs>\hello_world\src-gen\hw\hw.py", line 8, in <module>
    from LinguaFrancahw import (
      Tag, action_capsule_t, port_capsule, request_stop, start
    )
ImportError: DLL load failed while importing LinguaFrancahw: The specified module
could not be found.
```

To me, the error seemed to indicate some missing dependencies while importing LinguaFrancahw. “hw.py” was the top-level Python script generated by LF which was loading the reactor definitions (LinguaFrancahw) from “LinguaFrancahw.pyd”, and failing in the process. LF writes this top-level .py script and the .pyd file containing reactor definitions in the same directory by default.

I used the dependency walker to verify dependencies of the generated “LinguaFrancahw.pyd” file, but didn’t find any missing dependency libraries.

Note: The .pyd file generated when compiling a Python program on Windows is equivalent to the .so file generated on macOS or Linux.

Since Lingua Franca is primarily developed and tested on Unix-like systems, eventually, I concluded that the support on Windows wasn’t good enough. Therefore, I decided to run LF on Linux instead.

1.2 Working with LF on Linux

My first thought was to configure a Linux VM (VmWare Workstation or Oracle VirtualBox) on my Windows 11. But, after referring to some collateral, I decided against it. To run the Linux flavor of LF, I required basic virtualization only. Therefore, I decided to use Windows Subsystem for Linux

(WSL). WSL runs Ubuntu by default and integrates smoothly with VS Code (through an extension).

LF worked seamlessly for me using WSL on Windows without any issues. Refer to the next section for my detailed notes on working with WSL on windows and running LF on it.

2 Enable WSL2 on Windows 11, install Ubuntu Linux, and run Lingua Franca

2.1 Enable WSL2 on Windows 11 laptop

2.1.1 Check System Requirements

- Ensure your Windows 11 is up to date (build 22000 or later, which is standard for Windows 11).
- Verify your system supports virtualization:
 - Open Task Manager (Ctrl + Shift + Esc), go to the “Performance” tab, and check if “Virtualization” is listed as “Enabled” under the CPU section.
 - If not, you may need to enable virtualization in your BIOS/UEFI settings (consult your motherboard manual).

2.1.2 Enable Windows Subsystem for Linux (WSL)

- Open the Start menu, search for “Turn Windows features on or off”, and click it.
- In the “Windows Features” window, scroll down and check the box for Windows Subsystem for Linux.
- Click OK, then restart your computer when prompted.

2.1.3 Enable Virtual Machine Platform

- Return to the “Windows Features” window (repeat the search for “Turn Windows features on or off”).
- Check the box for Virtual Machine Platform.
- Click OK and restart your computer again if prompted.

2.1.4 Update WSL2 and set as the Default Version

- Open PowerShell or Command Prompt as Administrator:
 - Press Win + S, type PowerShell, right-click, and select “Run as administrator”.
- Run the following command to ensure WSL2 is installed fully and up-to-date: *PowerShell > wsl -update*
- Run the following command to ensure WSL2 is set as the default: *PowerShell > wsl -set-default-version 2*

This ensures new Linux distributions you install will use WSL2.

2.1.5 Verify WSL2 is Running

- In PowerShell, run:
PowerShell > wsl -list -verbose

Ensure your installed distribution (e.g., Ubuntu) shows 2 under the “VERSION” column, confirming WSL2 is active.

2.2 Install and setup Ubuntu on WSL2

2.2.1 Install a Linux Distribution (e.g., Ubuntu) via command line

In PowerShell (as Administrator), run:

```
PowerShell > wsl -install -d Ubuntu
```

This command installs Ubuntu 24.04 LTS and sets it up automatically. If you want a specific version, list available distributions with: *PowerShell > wsl -list -online*

Then install your preferred one, e.g., `wsl -install -d Ubuntu-22.04`.

2.2.2 Set Up Ubuntu

- After installation, launch Ubuntu from the Start menu or by typing `wsl` in PowerShell.
- The first launch will prompt you to create a default user account and password for the Linux environment.
- Next, update the package list in Ubuntu:
Linux > sudo apt update && *sudo apt upgrade -y*
- To verify the Ubuntu version:

```
Linux> lsb_release -a
No LSB modules are available.
Distributor ID: Ubuntu
Description:    Ubuntu 24.04.3 LTS
Release:        24.04
Codename:       noble
```

2.3 Install Lingua Franca pre-requisites in Ubuntu

2.3.1 Update Package Lists

```
Linux > sudo apt update
```

2.3.2 Install Java 17 (or later), gcc, g++, cmake, python3

```
Linux > sudo apt install openjdk-17-jdk
```

```
Linux > sudo apt install build-essential // installs gcc, g++ and make
OR
```

```
Linux > sudo apt install gcc
```

```
Linux > sudo apt install g++
```

```
Linux > sudo apt install cmake
```

```
Linux > sudo apt install -y python3 python3-pip python3-venv
```

All the above get installed under `/usr/bin` which is already in Linux PATH.

3 Install Lingua Franca command line

```
Linux > mkdir lf_install
```

```
// Get LF tar ball
```

```
Linux> wget <link of Linux x86_64 archive from Assets on LF page https://github.com/lf-lang/lingua-franca/releases>
```

Example:

```
Linux> wget https://github.com/lf-lang/lingua-franca/releases/download/nightly/
lf-cli-0.10.2-SNAPSHOT-nightly-20250925050324-Linux-x86_64.tar.gz
```

// Untar LF tar ball

```
Linux> tar xvf lf-cli-0.10.2-SNAPSHOT-nightly-20250925050324-Linux-x86_64.tar.gz
Linux> rm lf-cli-0.10.2-SNAPSHOT-nightly-20250925050324-Linux-x86_64.tar.gz
```

// Add soft links of LF binaries in /usr/local/bin because it is already in Linux PATH

```
Linux> sudo ln -s ~/lf_install/lf-cli-0.10.2-SNAPSHOT-nightly-20250925050324-Linux-x86_64/
bin/lfc /usr/local/bin/
Linux> sudo ln -s ~/lf_install/lf-cli-0.10.2-SNAPSHOT-nightly-20250925050324-Linux-x86_64/
bin/lfld /usr/local/bin/
Linux> sudo ln -s ~/lf_install/lf-cli-0.10.2-SNAPSHOT-nightly-20250925050324-Linux-x86_64/
bin/lff /usr/local/bin/
```

```
Linux> lfc --version
lfc 0.10.2-SNAPSHOT 7836d4be5 2025-09-25
```

// Make the data directory for your LF projects

```
Linux> mkdir -p ~/mydata/lf_work/myprojs
```

3.1 Use VS Code on Windows to connect with WSL2, and run Lingua Franca on it (in the Linux environment)

- Run VS Code on Windows
- Click "Open a Remote Window" button "><" at the bottom left of VS Code, followed by "Connect to WSL" which is now showing at the top-middle of VS Code
 - Above remote connection can be closed by again clicking on the ">< WSL: Ubuntu" button in the bottom of VS Code, and then selecting "Close Remote Connection" which is now showing at the top-middle of VS Code

After connecting to WSL, make sure the following extensions are installed in VS Code (need to be installed one time only)

- Remote WSL extension "WSL" in VS Code to enable it to open any folder in WSL sub-system, and build Linux applications
- Lingua Franca extension "Lingua Franca"

Once the above extensions are available, open the LF projects as usual from "File->Open Folder.." menu in VS Code, build/ run LF code as usual